灵熙智能 · AI进阶课

第二讲 · 60分钟

从预测下一个token，到会推理、会行动的Agent

机器怎样学会语言，并把一次回答扩展成可验证的行动闭环

Representation　Prediction　Alignment　Action



2012以后，搜索、规则、学习没有消失，只是重新分工

2012–16 学会表示
2017 建立关系
2018–22 生成与对齐
2024–26 推理与行动
未来 环境与科学

学习

哪些模式写不完?

理解语言、图像与上下文

搜索

下一步先试什么?

推理、规划、候选行动

规则

什么必须精确?

权限、流程、完成标准

环境反馈：执行以后发生了什么?

现代AI不是全新魔法，而是把历史思想重新组合，并放大到新的规模。

能力闭环不断扩大：每一步解决旧瓶颈，也制造新边界

2012—16 学会表示
2017 建立关系
2018—22 生成与对齐
2024—26 推理与行动
未来 环境与科学

2012

Representation

解决 自己学习怎么看

新边界 难解释、吃数据

2017

Attention

解决 动态建立关系

新边界 计算成本上升

2018—22

Pre-training + RLHF

解决 通用底座变助手

新边界 流畅不等于真实

2024—26

Reasoning + Agent

解决 多步探索并行动

新边界 错误产生现实后果

未来

World Model

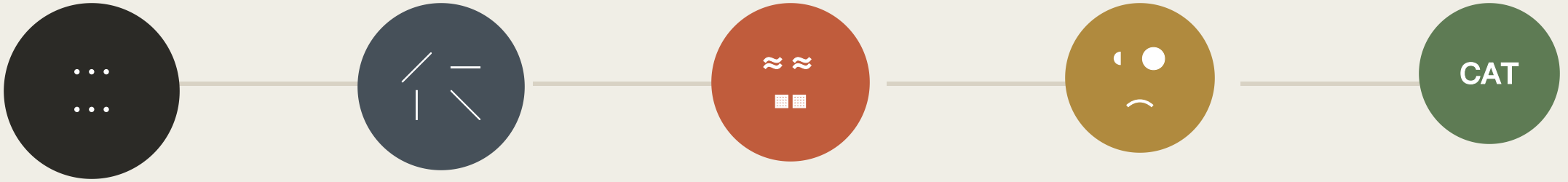
解决 行动前模拟后果

新边界 模拟不等于现实

AI史不是能力直线叠加，而是瓶颈从“不会看”迁移到“不会做、做错会怎样”。

深度学习的关键不只是“层数更多”，而是机器自己学习怎么看数据

2012–16 学会表示
2017 建立关系
2018–22 生成与对齐
2024–26 推理与行动
未来 环境与科学



像素
原始输入

边缘
方向与对比

纹理
重复局部模式

部件
眼睛、轮廓、形状

物体
对任务有用的表示

人提供：原始数据 + 目标 + 反馈

网络学习：对任务有用的内部表示

会分类不等于像人一样理解；表示仍受训练数据、目标和偏差限制。

承上｜能力链从表示开始

下一步｜看AlphaGo怎样把学习重新接回搜索04

现代突破不是学习替代搜索，而是网络、搜索与反馈组成闭环



NATURE · 2016

doi:10.1038/nature16961

Mastering the game of Go with deep neural networks and tree search

David Silver[1]*, Aja Huang[1]*, Chris J. Maddison[1], Arthur Guez[1], Laurent Sifre[1], George van den Driessche[1], Julian Schrittwieser[1], Ioannis Antonoglou[1], Veda Panneershelvam[1], Marc Lanctot[1], Sander Dieleman[1], Dominik Grewe[1], John Nham[2], Nal Kalchbrenner[1], Ilya Sutskever[2], Timothy Lillicrap[1], Madeleine Leach[1], Koray Kavukcuoglu[1], Thore Graepel[1] & Demis Hassabis[1]

The game of Go has long been viewed as the most challenging of classic games for artificial intelligence owing to its enormous search space and the difficulty of evaluating board positions and moves. Here we introduce a new approach to computer Go that uses 'value networks' to evaluate board positions and 'policy networks' to select moves. These deep neural networks are trained by a novel combination of supervised learning from human expert games, and reinforcement learning from games of self-play. Without any lookahead search, the neural networks play Go at the level of state-of-the-art Monte Carlo tree search programs that simulate thousands of random games of self-play. We also introduce a new search algorithm that combines Monte Carlo simulation with value and policy networks. Using this search algorithm, our program AlphaGo achieved a 99.8% winning rate against other Go programs, and defeated the human European Go champion by 5 games to 0. This is the first time that a computer program has defeated a human professional player in the full-sized game of Go, a feat previously thought to be at least a decade away.

All games of perfect information have an optimal value function, $v^*(s)$, which determines the outcome of the game, from every board position or state s, under perfect play by all players. These games may be solved by recursively computing the optimal value function in a search tree containing approximately b^d possible sequences of moves, where b is the game's breadth (number of legal moves per position) and d is its depth (game length). In large games, such as chess ($b \approx 35$, $d \approx 80$)[1] and especially Go ($b \approx 250$, $d \approx 150$)[1], exhaustive search is infeasible[2,3], but the effective search space can be reduced by two general principles. First, the depth of the search may be reduced by position evaluation: truncating the search tree at state s and replacing the subtree below s by an approximate value function $v(s) \approx v^*(s)$ that predicts the outcome from state s. This approach has led to superhuman performance in chess[4], checkers[5] and othello[6], but it was believed to be intractable in Go due to the complexity of the game[7]. Second, the breadth of the search may be reduced by sampling actions from a policy $p(a|s)$ that is a probability distribution over possible moves a in position s. For example, Monte Carlo rollouts[8] search to maximum depth without branching at all, by sampling long sequences of actions for both players from a policy p. Averaging over such rollouts can provide an effective position evaluation, achieving superhuman performance in backgammon[8] and Scrabble[9], and weak amateur level play in Go[10].

Monte Carlo tree search (MCTS)[11,12] uses Monte Carlo rollouts to estimate the value of each state in a search tree. As more simulations are executed, the search tree grows larger and the relevant values become more accurate. The policy used to select actions during search is also improved over time, by selecting children with higher values. Asymptotically, this policy converges to optimal play, and the evaluations converge to the optimal value function[12]. The strongest current Go programs are based on MCTS, enhanced by policies that are trained to predict human expert moves[13]. These policies are used to narrow the search to a beam of high-probability actions, and to sample actions during rollouts. This approach has achieved strong amateur play[13–15]. However, prior work has been limited to shallow policies[13–15] or value functions[16] based on a linear combination of input features.

Recently, deep convolutional neural networks have achieved unprecedented performance in visual domains: for example, image classification[17], face recognition[18], and playing Atari games[19]. They use many layers of neurons, each arranged in overlapping tiles, to construct increasingly abstract, localized representations of an image[20]. We employ a similar architecture for the game of Go. We pass in the board position as a 19×19 image and use convolutional layers to construct a representation of the position. We use these neural networks to reduce the effective depth and breadth of the search tree: evaluating positions using a value network, and sampling actions using a policy network.

We train the neural networks using a pipeline consisting of several stages of machine learning (Fig. 1). We begin by training a supervised learning (SL) policy network p_σ directly from expert human moves. This provides fast, efficient learning updates with immediate feedback and high-quality gradients. Similar to prior work[13,15], we also train a fast policy p_π that can rapidly sample actions during rollouts. Next, we train a reinforcement learning (RL) policy network p_ρ that improves the SL policy network by optimizing the final outcome of games of self-play. This adjusts the policy towards the correct goal of winning games, rather than maximizing predictive accuracy. Finally, we train a value network v_θ that predicts the winner of games played by the RL policy network against itself. Our program AlphaGo efficiently combines the policy and value networks with MCTS.

Supervised learning of policy networks

For the first stage of the training pipeline, we build on prior work on predicting expert moves in the game of Go using supervised learning[13,21–24]. The SL policy network $p_\sigma(a|s)$ alternates between convolutional layers with weights σ, and rectifier nonlinearities. A final softmax layer outputs a probability distribution over all legal moves a. The input s to the policy network is a simple representation of the board state (see Extended Data Table 2). The policy network is trained on randomly

[1]Google DeepMind, 5 New Street Square, London EC4A 3TW, UK. [2]Google, 1600 Amphitheatre Parkway, Mountain View, California 94043, USA.
*These authors contributed equally to this work.

© 2016 Macmillan Publishers Limited. All rights reserved

神经网络
给方向、评估局面

树搜索
探索未来后果

自我对弈
产生新经验

系统能力来自组合：学习缩小搜索范围，搜索检查后果，反馈继续改进学习。

承上 | 机器开始自己学习表示

下一步 | 把问题从棋盘转向更模糊的语言05

当前这个“它”，到底应该联系前面的谁?

奖杯放不进箱子，因为它太大了。



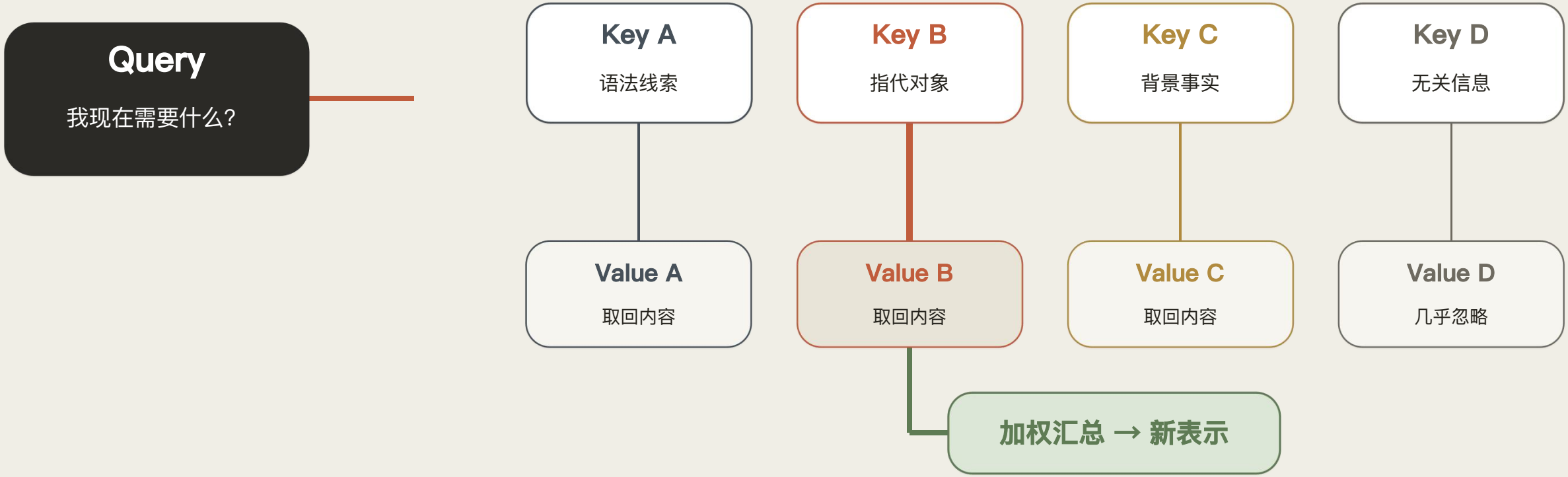
奖杯放不进箱子，因为它太小了。



语言不是给每个词查固定定义，而是根据当前上下文重新建立关系。

Attention：当前这一步，动态决定哪些上下文最相关

2012–16 学会表示 | **2017 建立关系** | 2018–22 生成与对齐 | 2024–26 推理与行动 | 未来 环境与科学



Query / Key / Value只是检索类比；Attention找到关联，不自动保证因果正确。

承上 | 词义来自上下文关系

下一步 | Transformer把信息交换变成可扩展骨架07

Transformer提供了可堆叠、可并行训练的“信息交换骨架”

2012—16 学会表示 | **2017 建立关系** | 2018—22 生成与对齐 | 2024—26 推理与行动 | 未来 环境与科学

TRANSFORMER · FIGURE 1

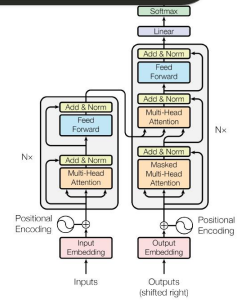


Figure 1: The Transformer - model architecture.

The Transformer follows this overall architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder, shown in the left and right halves of Figure 1, respectively.

3.1 Encoder and Decoder Stacks

Encoder: The encoder is composed of a stack of $N = 6$ identical layers. Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position-wise fully connected feed-forward network. We employ a residual connection [11] around each of the two sub-layers, followed by layer normalization [1]. That is, the output of each sub-layer is $\mathrm{LayerNorm}(x + \mathrm{Sublayer}(x))$, where $\mathrm{Sublayer}(x)$ is the function implemented by the sub-layer itself. To facilitate these residual connections, all sub-layers in the model, as well as the embedding layers, produce outputs of dimension $d_{\mathrm{model}} = 512$.

Decoder: The decoder is also composed of a stack of $N = 6$ identical layers. In addition to the two sub-layers in each encoder layer, the decoder inserts a third sub-layer, which performs multi-head attention over the output of the encoder stack. Similar to the encoder, we employ residual connections around each of the sub-layers, followed by layer normalization. We also modify the self-attention sub-layer in the decoder stack to prevent positions from attending to subsequent positions. This masking, combined with fact that the output embeddings are offset by one position, ensures that the predictions for position i can depend only on the known outputs at positions less than i.

3.2 Attention

An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors. The output is computed as a weighted sum

3

Multi-head Attention — **Feed-forward Network**

一层：位置之间交换信息 + 每个位置独立加工

重复堆叠 × N

更直接的远距离关系 + 更适合并行训练，让同一骨架可以随数据与算力扩展。

承上 | Attention决定当前该看哪里

下一步 | 架构有了，模型到底靠什么目标学习08

一个看似简单的训练目标，为什么会逼出复杂能力?

2012–16 学会表示 | 2017 建立关系 | **2018–22 生成与对齐** | 2024–26 推理与行动 | 未来 环境与科学

事实

法国的首都是 ____

可能答案：巴黎

常识

杯子从桌边掉下去，接下来最可能 ____

可能答案：落地

逻辑

所有员工已培训，小李是员工，所以小李 ____

可能答案：已培训

代码

if password_is_wrong: 下一行最可能 ____

可能答案：return / retry

要长期猜对下一步，模型必须压缩语法、事实模式、常识、逻辑与代码结构。

token = 模型处理文本时使用的片段单位，不严格等于一个字或一个单词。

承上 | Transformer给出可扩展骨架

下一步 | 看模型怎样逐token生成完整输出09

模型不是一次写完整段，而是把每次结果放回上下文继续预测

读取上下文 → 下一个token概率分布 → 选择 / 采样 → 放回上下文 → 继续预测

‘今天的会议重点是’ → 规划 0.42 | 复盘 0.25 | 预算 0.18 | 其他...

前面错了：错误也会进入后续上下文并放大

每步是概率选择：同一问题可能出现不同表达

流畅性来自生成机制，不自动提供事实保证；这也是ELIZA提醒的延续。

承上 | 预测目标压缩了大量文本规律

下一步 | 规模化预训练让一个底座适配多任务10

从“一个任务一个模型”，变成“一个基础模型适配多种任务”

GPT-3 · 2020



arXiv:2005.14165v4 [cs.CL] 22 Jul 2020

Language Models are Few-Shot Learners

Tom B. Brown* Benjamin Mann* Nick Ryder* Melanie Subbiah*
Jared Kaplan† Prafulla Dhariwal Arvind Neelakantan Pranav Shyam Girish Sastry
Amanda Askell Sandhini Agarwal Ariel Herbert-Voss Gretchen Krueger Tom Henighan
Rewon Child Aditya Ramesh Daniel M. Ziegler Jeffrey Wu Clemens Winter
Christopher Hesse Mark Chen Eric Sigler Mateusz Litwin Scott Gray
Benjamin Chess Jack Clark Christopher Berner
Sam McCandlish Alec Radford Ilya Sutskever Dario Amodei

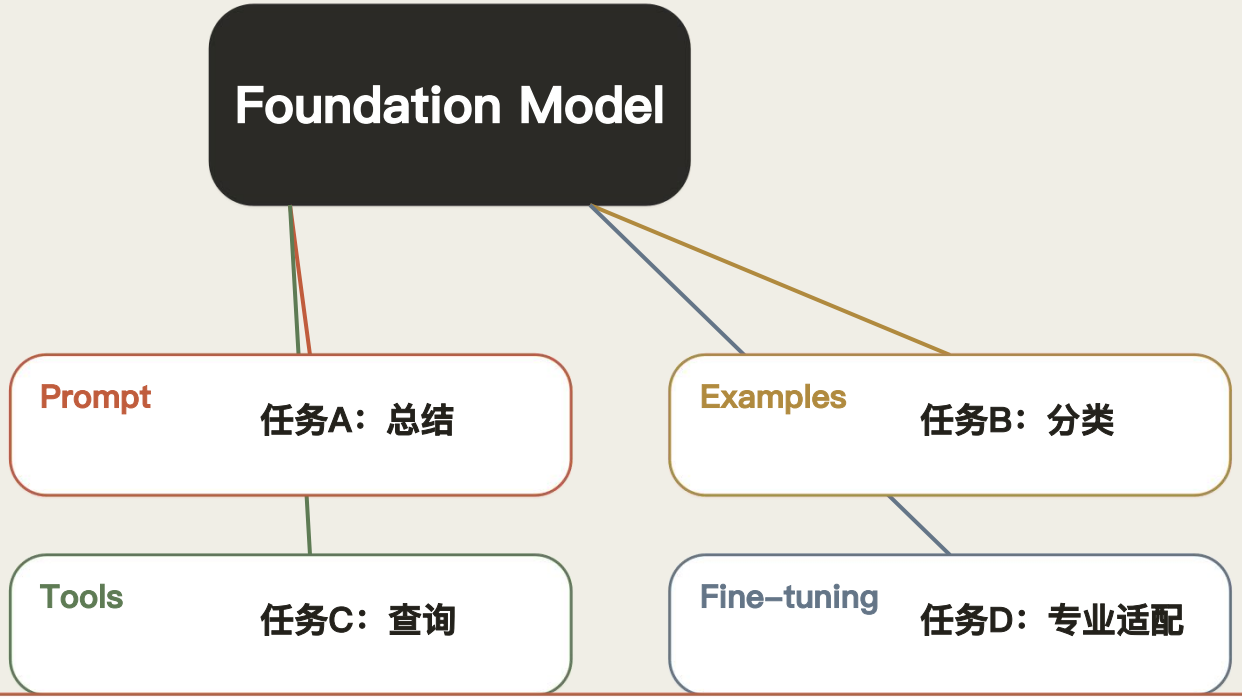
OpenAI

Abstract

Recent work has demonstrated substantial gains on many NLP tasks and benchmarks by pre-training on a large corpus of text followed by fine-tuning on a specific task. While typically task-agnostic in architecture, this method still requires task-specific fine-tuning datasets of thousands or tens of thousands of examples. By contrast, humans can generally perform a new language task from only a few examples or from simple instructions – something which current NLP systems still largely struggle to do. Here we show that scaling up language models greatly improves task-agnostic, few-shot performance, sometimes even reaching competitiveness with prior state-of-the-art fine-tuning approaches. Specifically, we train GPT-3, an autoregressive language model with 175 billion parameters, 10x more than any previous non-sparse language model, and test its performance in the few-shot setting. For all tasks, GPT-3 is applied without any gradient updates or fine-tuning, with tasks and few-shot demonstrations specified purely via text interaction with the model. GPT-3 achieves strong performance on many NLP datasets, including translation, question-answering, and cloze tasks, as well as several tasks that require on-the-fly reasoning or domain adaptation, such as unscrambling words, using a novel word in a sentence, or performing 3-digit arithmetic. At the same time, we also identify some datasets where GPT-3's few-shot learning still struggles, as well as some datasets where GPT-3 faces methodological issues related to training on large web corpora. Finally, we find that GPT-3 can generate samples of news articles which human evaluators have difficulty distinguishing from articles written by humans. We discuss broader societal impacts of this finding and of GPT-3 in general.

*Equal contribution
†Johns Hopkins University, OpenAI

Author contributions listed at end of paper.



预训练把多种任务规律压进同一套参数；通用接口不等于每个专业场景都可靠。

会续写不等于会协助：人类反馈把基础模型塑造成助手

Pre-training

学语言、知识与模式

Instruction Tuning

学问题—回答与任务格式

Human Preference

比较回答，学习有用与安全

INSTRUCTGPT · 2022



它证明了

对齐能显著改变用户偏好；较小对齐模型可胜过更大未对齐底座。

它没证明

人类偏好就是客观真理；模型更听话也不代表事实自动正确。

Alignment主要把“会生成”变成“会协助”，不能替代来源、规则与事实核查。

爆发不是一篇论文，而是六块技术与产品拼图同时成熟

2012–16 学会表示 · 2017 建立关系 · **2018–22 生成与对齐** · 2024–26 推理与行动 · 未来 环境与科学

01 Transformer
可扩展架构

02 Internet Data
通用文本

03 GPU Cluster
训练算力

04 Pre-training
通用底座

05 Instruction + RLHF
助手行为

06 Chat UI
零门槛交互

能力 × 低交互成本 × 高频反馈 = 大众化使用

聊天框没有凭空创造能力，但把提问、追问、纠正和继续变成几乎零学习成本。

承上｜底座经过指令与偏好塑形

下一步｜复杂问题为什么需要回答时多探索13

困难任务不只靠更大训练，也可以在回答时投入更多搜索与检查

Fast answer

简单、低风险问题

一次生成 → 输出

More inference-time compute

复杂、可验证或高风险问题

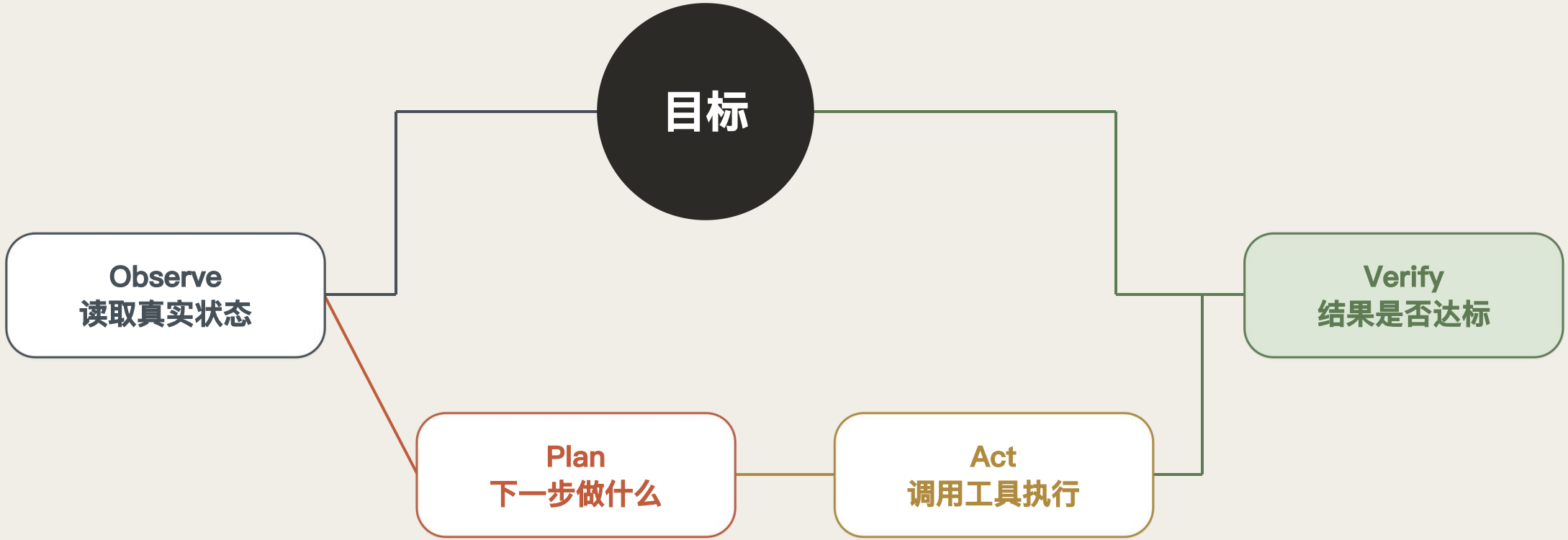
拆分 — 尝试 — 检查 — 修正

权衡：更可能解决难题，但更慢、更贵；推理过程看起来完整，仍不等于事实已验证。

搜索思想再次回来：探索候选步骤、比较结果、回退和修正。

从“给答案”到“完成任务”：关键是观察—行动—验证的可控闭环

2012–16 学会表示
2017 建立关系
2018–22 生成与对齐
2024–26 推理与行动
未来 环境与科学



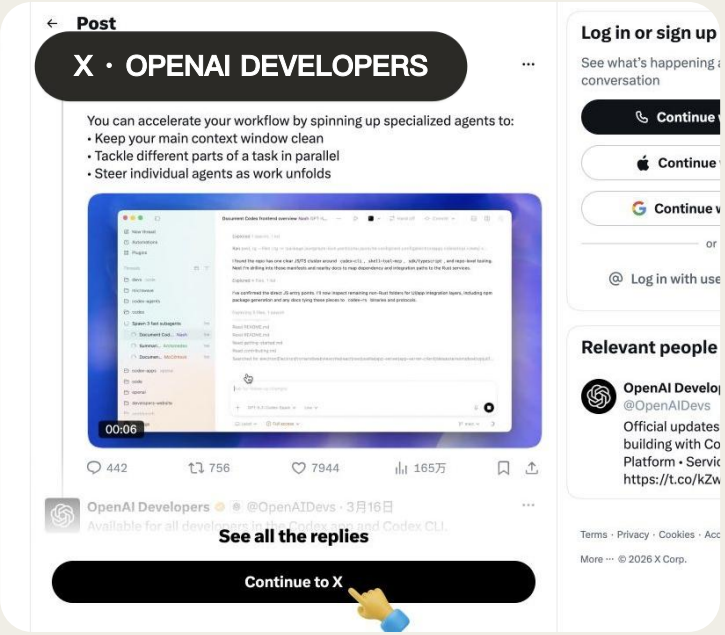
最小权限 | 明确完成标准 | 关键动作确认 | 日志与可追溯 | 失败停止 / 人工接管

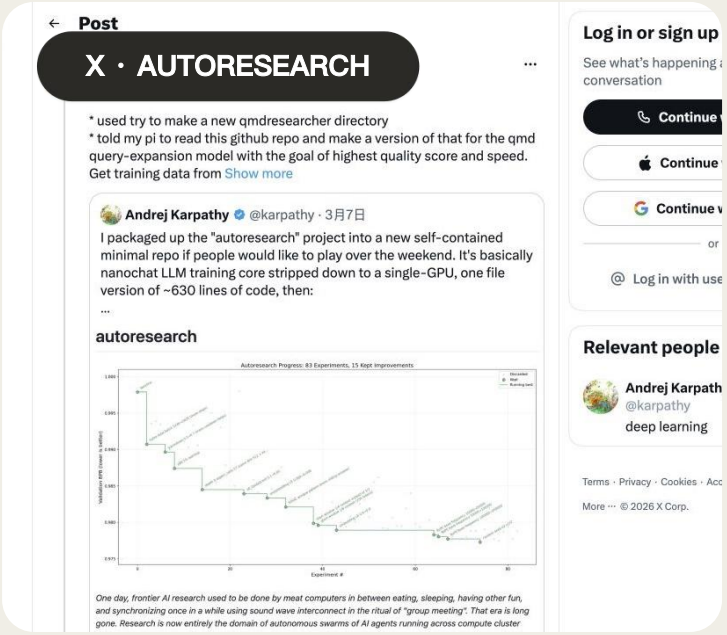
承上｜推理开始多步探索与检查

下一步｜单Agent怎样扩展成并行工作系统15

扩大执行规模的前提：任务可拆分、环境可运行、结果可验证

2012–16 学会表示 → 2017 建立关系 → 2018–22 生成与对齐 → **2024–26 推理与行动** → 未来 环境与科学





它证明了

独立角色可并行；在可运行、可量化环境中，Agent能反复提出改动并读取指标。

它没证明

多Agent天然正确；一个公开案例已证明AI能普遍独立完成科研。

主Agent裁决 | 研究找证据 | 实现产出 | 测试独立验证 | 审查寻找遗漏

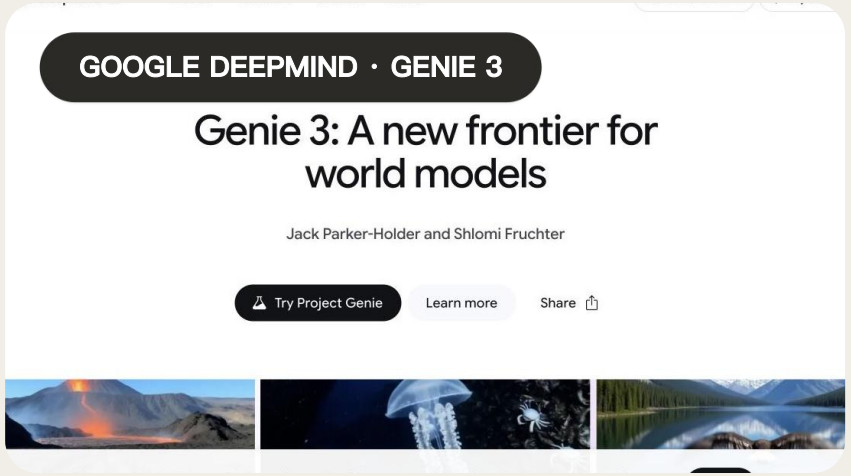
多个Agent共享同一错误前提，只会更快重复错误；独立证据和验收标准才是价值。

承上 | Agent形成观察—行动—验证闭环

下一步 | 现实试错昂贵时，能否先模拟后果16

AI走出聊天框后，关键不是生成得更像，而是能否得到可靠反馈

2012–16 学会表示 → 2017 建立关系 → 2018–22 生成与对齐 → 2024–26 推理与行动 → **未来** 环境与科学



当前状态 + 行动 → 内部模拟结果 → 真实行动

它证明了

生成可交互环境，显示Agent可在模拟环境中行动与学习的研究方向。

它没证明

生成环境等于真实世界；模拟成功能自动跨越物理、社会和极端情况差距。

AI for Science

模型预测候选 → 实验验证 → 可重复证据 → 反馈修正

AlphaFold 3提示：AI的重要历史也发生在蛋白质、DNA、RNA与小分子相互作用空间。

World Model需要现实校准；AI for Science需要实验闭环。预测不是终点。

承上 | 可运行环境让Agent反复实验

下一步 | 把两节课压缩成一条闭环演化史 17

AI一直在扩大“机器能独立完成的闭环”，风险也同步升级

2012–16 学会表示
2017 建立关系
2018–22 生成与对齐
2024–26 推理与行动
未来 环境与科学

规则	学习	表征	生成	对齐	推理	Agent
人写步骤	从错误调参数	自己学习怎么看	预测语言与任务	通过反馈学协助	更多搜索与检查	行动并验证

Fluency ≠ Truth	语言自然不能替代来源与核查
Reasoning ≠ Verification	过程完整不能替代结果验收
Agency ≠ Authority	会行动不等于可以放权
Breakthrough ≠ Single Trick	突破来自系统条件会合

开放问题：十年后回看，2026年的关键转折会是Reasoning、Agent、World Model，还是尚未出现的方法?

承上 | 未来能力取决于环境与实验反馈

下一步 | 用四条判断看待下一次AI浪潮18